

Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer¹ and Kristof Schröder²

August 2026

Abstract. The Method of Alternating Projections (MAP) is the canonical algorithm for estimating high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. The worker-firm match-count matrix records each observed link directly. Together with the worker and firm count diagonals, it forms a graph Laplacian after a sign flip and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver with a reusable preconditioner built from factor-pair subproblems, such as worker-firm and worker-year, that retain these match counts. Benchmarks show that all methods are fast on well-connected designs. As connectivity weakens, MAP and diagonally preconditioned LSMR slow down, while factor-pair preconditioning remains fast and its runtime falls because the graph preconditioner is cheaper to construct on sparser graphs. On a near-nested design with 10 million observations, factor-pair LSMR completes in 4.63 seconds, compared with 62.2 seconds for the fastest MAP implementation.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed effects appear in roughly half of the research published in leading economics and finance journals, according to Goldsmith-Pinkham (2026). Across applied economics, labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual, physician, and region fixed effects in mover designs; and education researchers use school, student, teacher, and student-teacher effects.

In each case, computation usually proceeds by absorbing the fixed effects before estimating the coefficients of interest. The Frisch-Waugh-Lovell (FWL) theorem first removes the variation explained by the fixed effects from the outcome and regressors; a second, low-dimensional regression then relates the residualized outcome to the residualized covariates (Frisch and Waugh 1933; Lovell 1963). With high-dimensional fixed effects, software usually performs the residualization with the Method of Alternating Projections

¹trivago

²appliedAI Institute for Europe gGmbH

(MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Packages including Stata’s `reghdfe` (Correia 2014; 2017), R’s `fixest` (Bergé et al. 2026), and Python’s `PyFixest` (PyFixest Developers 2026) use MAP or accelerated variants of it.

MAP updates one fixed-effect dimension at a time. In the worker-firm wage model of Abowd et al. (1999), extended here with year effects, a pass subtracts worker, firm, and year means in sequence; the firm update therefore receives the residual left by the worker update rather than jointly solving the worker-firm problem.

That distinction matters when workers rarely move between groups of firms. Movers create the paths that separate worker effects from firm wage premia, while stayers add observations without connecting firms (Correia 2017; Jochmans and Weidner 2019). With few paths between groups, a change on one side of the graph reaches the other side only through successive MAP passes. The mobility pattern therefore affects identification, precision, and computation.

We follow Jochmans and Weidner (2019) and call the second-smallest eigenvalue of the normalized Laplacian the *Gap*. For factor pair (q, r) , let L_{qr} be the weighted graph Laplacian and Δ_{qr} its degree matrix. Then

$$S_{qr} = \Delta_{qr}^{-\frac{1}{2}} L_{qr} \Delta_{qr}^{-\frac{1}{2}}, \quad \text{Gap}_{qr} = \lambda_2(S_{qr}). \quad (1)$$

Edge weights are observed co-occurrence counts, and a larger Gap indicates stronger connectivity between the two fixed-effect dimensions. Our empirical results report the worker-firm Gap.³

The off-diagonal blocks of the fixed-effect cross product, or Gramian, record these worker-firm links, while its diagonal blocks contain worker, firm, and year observation counts (Correia 2017). MAP accesses the match-count blocks only through successive residuals; our preconditioner instead groups them with the corresponding count blocks by factor pair. A sign change turns each pair block into a graph Laplacian, and sparse approximate Cholesky methods supply local corrections at low cost (Spielman and Teng 2014; Gao et al. 2025). A weighted sum of these local corrections preconditions LSMR.⁴

The pair factorizations have a setup cost, so they are unlikely to help when MAP already converges in a few passes. On a weakly connected graph, however, they can save substantial iteration time. Figure 1 reports total regression time as mobility and sorting change: the left column compares package defaults, whereas the right holds the package and achieved accuracy fixed within PyFixest using tolerances calibrated in Section 7.

³We compute the Gap after iterative singleton removal. If the remaining graph is disconnected, we use the component containing the most retained observations and report its observation share; Section 7 gives the full convention.

⁴`FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026) also uses LSMR (Fong and Saunders 2011), with diagonal preconditioning based on fixed-effect counts, whereas the factor-pair preconditioner also uses links between fixed-effect dimensions.

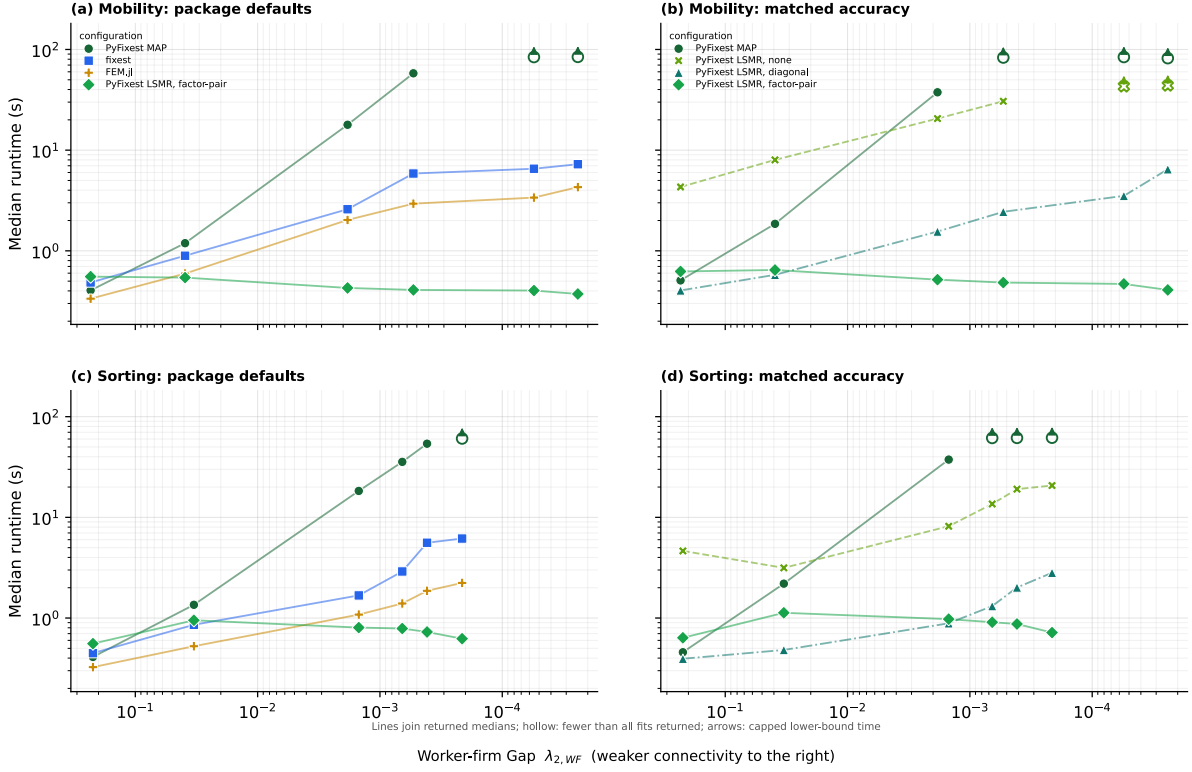


Figure 1: Median regression time against the worker-firm Gap $\lambda_{2,WF}$, on logarithmic axes. The Gap is computed after singleton removal on the connected component containing the most observations; larger values mean stronger connectivity, and the shared reversed horizontal scale makes connectivity weaken from left to right. Each simulated worker-firm-year panel has 1 million observations, with worker mobility varying in the top row and sorting among movers in the bottom row. The left column compares package defaults, including each package’s singleton rule, while the right compares four PyFixest configurations at tolerances that give comparable coefficient and residual errors in Section 7. A filled marker means all three planned fits returned estimates; a hollow marker means one or two did. Lines join observed medians rather than fitted trends, and an arrow marks the median time at the iteration cap, a lower bound on completion time; other failures are omitted.

All implementations finish quickly at high Gap values. MAP and unpreconditioned LSMR slow sharply as the Gap falls; factor-pair LSMR remains fast, and its runtime falls in the lowest-mobility designs because the local graphs become cheaper to factorize.

The next four sections introduce fixed-effect absorption, the AKM example, the Gramian, and the link between MAP and graph connectivity. Section 6 develops the factor-pair preconditioner. Sections 7 and 8 report the benchmarks and software interface, Section 9 concludes, and the appendix compares memory use.

2. Absorbing Fixed Effects⁵

We study the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (2)$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. Because D may have hundreds of thousands or millions of columns in high-dimensional applications, forming $[X \ D]$ or inverting its cross product can be computationally infeasible. The Frisch-Waugh-Lovell (FWL) theorem lets us compute $\hat{\beta}$ without either operation.

⁵Researchers employ several names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

FWL removes the fixed-effect means from y and each column of X , then regresses the residualized outcome on the residualized covariates. We write M_D for the operation that removes those means; in matrix terms, it is the weighted residual-making projection for the fixed-effect indicators, so $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$. The resulting low-dimensional regression gives

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = (\tilde{X}' W \tilde{X})^{-1} \tilde{X}' W \tilde{y}, \quad (3)$$

where W is a diagonal matrix of weights. With one covariate, this amounts to three regressions: y on D , x on D , and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D .

To residualize the outcome and each covariate in X , we project them onto the column space of the fixed-effect dummy matrix D by weighted least squares. For a variable μ to be residualized - either y or one column of X - we solve

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \| D\alpha - \mu \|_W^2, \quad (4)$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 4,

$$D'W(D\hat{\alpha}_\mu - \mu) = 0, \quad G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD, \quad (5)$$

determines $\hat{\alpha}_\mu$. Its coefficient matrix G is the same for each FWL residualization; only the variable being residualized changes, so the computational cost depends on the structure of the Gramian G , the weighted cross-product matrix of the fixed-effect dummies. In the AKM worker-firm model, this structure has a concrete interpretation: worker moves link firms through shared workers and determine the off-diagonal blocks of G .

3. A Running Example: The AKM Model

Throughout the paper, we use the AKM model of Abowd et al. (1999) as the worker-firm example, in which workers observed at more than one firm provide the comparisons needed to distinguish persistent worker heterogeneity from firm wage premia. We write the regression as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (6)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

The AKM specification can be represented as a bipartite graph, with workers and firms as nodes and employment spells as edges. Worker moves induce a firm-to-firm graph by linking firms that employ the same worker; stayers - workers who never change employer - add observations to existing worker-firm links but do not bridge firms. Year effects enter as a third, low-dimensional factor on the same records, and Figure 2 contrasts a well-connected mobility graph with one that fragments under strong sorting.

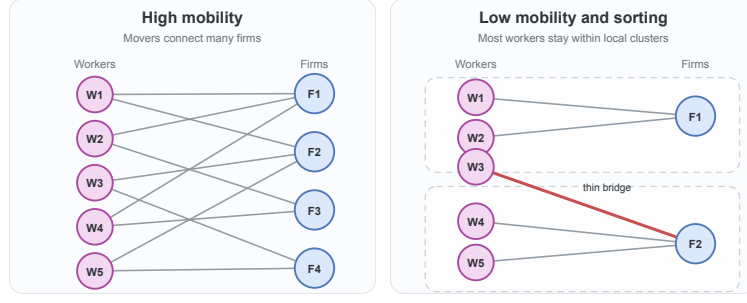


Figure 2: Worker-firm graphs under high and low mobility. High mobility creates many paths between firms, whereas low mobility and strong sorting leave only a few worker moves connecting otherwise separate groups.

A worker observed at one firm provides no comparison between worker productivity and the firm wage premium. Wages for the same worker at different firms do separate the two effects, especially when the worker moves between groups that otherwise share few employees; these comparisons appear both as edges in the mobility graph and as entries in the worker-firm cross-tabulation of the Gramian.

4. The Graph Structure of the Gramian

The block structure of the Gramian $G = D'WD$ represents the bipartite graph of worker and firm connections introduced in Section 3 (Correia 2017). Ordering the columns of D as worker, firm, and year levels gives

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (7)$$

Blue marks the **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} , which contain weighted counts for workers, firms, and years. Because an observation belongs to one level of each factor, these blocks are diagonal and can be inverted by dividing by group counts.

The **off-diagonal blocks** are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j , and the worker-year and firm-year blocks have analogous interpretations.

The small worker-firm panel below illustrates the construction with all regression weights set to one, so $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

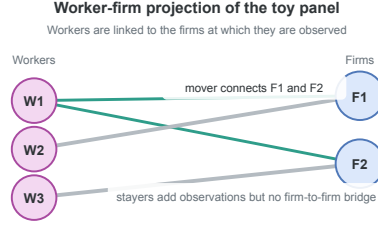


Figure 3: Worker-firm graph for the example panel. Worker W_1 connects the firms by working at both, while W_2 works only at F_1 and W_3 only at F_2 .

Worker W_1 supplies the only link between F_1 and F_2 in Figure 3, whereas workers W_2 and W_3 remain at one firm. Because each worker is observed twice, each firm three times, and each year three times, the diagonal count matrices are

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (8)$$

For the worker-firm pair, the off-diagonal cross-tabulation is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (9)$$

The first row records the mover's two matches, while the remaining rows record the two stayers. The worker-year and firm-year cross-tabulations are

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (10)$$

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, these count and cross-tabulation blocks form the full Gramian,

$$G = \begin{pmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{pmatrix}. \quad (11)$$

The worker-firm submatrix contains worker and firm counts on its diagonal and match counts in C_{WF} . After changing the sign of C_{WF} , the off-diagonal entries are non-positive and each row sums to zero because its diagonal observation count equals the sum of its match counts. The submatrix is therefore the weighted graph Laplacian

$$L_{WF} = \begin{pmatrix} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{pmatrix}. \quad (12)$$

Every factor pair has a Laplacian of this form. Section 6 uses these matrices in the preconditioner; MAP uses only their diagonal count blocks in each update.

5. Alternating Projections and Graph Connectivity

The Method of Alternating Projections (MAP), also called iterative demeaning or the “zig-zag” algorithm, is the standard method for absorbing several fixed effects (Guimaraes and Portugal 2010; Gaure 2013). Implementations often accelerate the basic iteration (Berge 2018; Correia 2017); `fixest`, for example, uses Irons-Tuck extrapolation (Irons and Tuck 1969; Bergé et al. 2026).

In the worker-firm-year model, a MAP pass starts with a partial residual, subtracts its mean within each worker, and then computes firm and year means from the successively updated residual. Repeating the pass eventually removes all three sets of means.

With $D = [D_W \ D_F \ D_Y]$, the FWL normal equations in Equation 5 are

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_W' W \mu \\ D_F' W \mu \\ D_Y' W \mu \end{pmatrix}. \quad (13)$$

Holding the current firm and year effects fixed, the worker update uses the corresponding row of this system,

$$G_{WW} \alpha_W = D_W' W (\mu - D_F \alpha_F - D_Y \alpha_Y). \quad (14)$$

Because the diagonal entries of G_{WW} are workers’ total observation weights, dividing the right-hand side by those counts gives the weighted worker means. Firm and year updates have the same form, and one complete MAP pass applies all three inexpensive group-mean calculations in sequence.

The match-count blocks therefore enter only through the changing partial residual; MAP never solves a worker-firm, worker-year, or firm-year block jointly.

With many overlapping employment histories, a worker update changes residuals at many firms, and the next firm update carries that information onward. Sparse mobility, strong sorting, or near nesting confines the changes to a narrower part of the graph, so MAP may need many passes even though each pass remains cheap.

The factor-pair Gap $\lambda_{2,qr}$ summarizes the strength of these links, and a small value warns that information may move slowly across the pair graph. In a model with three or more fixed effects, however, the Gap remains a diagnostic for one pair rather than a bound on the convergence rate of the full model.

6. The Factor-Pair Schwarz Preconditioner

6.1. LSMR and Preconditioners

Factor-pair preconditioning incorporates cross-factor links into each LSMR iteration without changing the least-squares problem.

LSMR solves the fixed-effect least-squares problem in Equation 4 by multiplying vectors by D and D' rather than forming the Gramian, while a preconditioner uses a cheap approximation to G^{-1} to rescale the search directions. Because LSMR still checks convergence against the original least-squares problem,

this approximation changes the number of iterations rather than the residualized variables reached at convergence.

The exact inverse G^{-1} provides a useful benchmark. If $M^{-1} = G^{-1}$, the preconditioned normal equation is the identity,⁶

$$M^{-1}G = G^{-1}G = I. \quad (15)$$

One correction would then suffice, but computing G^{-1} costs as much as the original fixed-effect problem. A useful approximation must capture the poorly determined directions while remaining cheap to construct and apply.⁷

6.2. Diagonal Preconditioning

The simplest approximation drops every cross-factor block and inverts only the group counts,

$$M_{\text{diag}}^{-1} = \text{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \quad (16)$$

Each block is diagonal, so applying M_{diag}^{-1} amounts to dividing worker, firm, and year entries by their total observation weights. `FixedEffectModels.jl` uses this preconditioner (Fong and Saunders 2011; `FixedEffectModels.jl` Developers 2026).

Count scaling is cheap and can work well on a well-connected graph. It gives the same scale factor to two equally large firms even if one shares workers broadly and the other belongs to a nearly isolated group. LSMR continues to update against the original system, so the omitted match-count terms return through later matrix-vector products. A weak diagonal approximation therefore increases the number of outer iterations without changing the least-squares target.

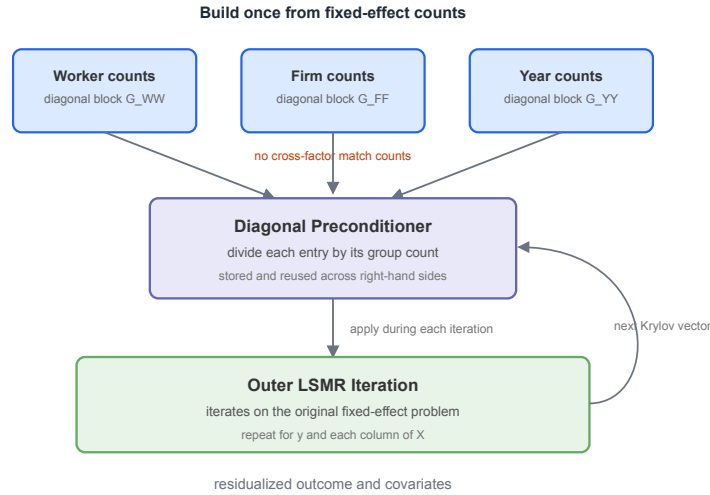


Figure 4: Diagonal preconditioning within LSMR. Worker, firm, and year counts are computed once and reused for the outcome and covariates; at each iteration, the preconditioner divides factor-level entries by their total observation weights. Because LSMR continues to check the original fixed-effect least-squares problem, diagonal scaling affects convergence speed rather than the residualized variables reached at convergence.

⁶As in any model with several fixed effects, the level effects are pinned down only up to a normalization: we can add a constant to every worker effect and subtract it from every firm effect without changing the fitted values $D\alpha$, and likewise for years. The dummy-coded $G = D'WD$ and the smaller blocks inverted below are therefore singular until we impose a normalization. We impose one normalization for the worker-firm pair and a second when year effects are added, as in the Section 4 example. We adopt the standard normalization within each connected set and read every inverse below as that of the resulting system. The estimated effects depend on the normalization; the residualized outcome and regressors, which are all the regression uses, do not.

⁷LSMR never constructs $M^{-1}G$ or G explicitly; it multiplies vectors by D and D' and applies M^{-1} .

6.3. What Diagonal Scaling Omits

The worker-firm block inverse makes the omitted terms explicit: **blue** marks the count-only terms used by diagonal preconditioning, whereas **orange** marks terms that depend on cross-factor links. For the worker-firm-year model,

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (17)$$

Restricting G to the worker and firm dimensions gives

$$G_{WF} = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (18)$$

whose inverse is

$$G_{WF}^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1} C_{WF} S^{-1} C_{(WF)'} G_{WW}^{-1} & -G_{WW}^{-1} C_{WF} S^{-1} \\ -S^{-1} C_{(WF)'} G_{WW}^{-1} & S^{-1} \end{pmatrix}. \quad (19)$$

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \quad (20)$$

The G_{WW}^{-1} term gives the worker-count correction used by diagonal preconditioning. The remaining terms depend on the Schur complement S , which compares firms after accounting for their shared workers and couples the worker and firm corrections. Factoring S is expensive: exact elimination can add many nonzero entries, with time and memory determined by component size and fill-in. Sparse mobility can make diagonally preconditioned LSMR slow while making a local factorization cheaper because there are fewer links. With three factors, the full inverse can involve all three pairwise cross-tabulations.

6.4. The Factor-Pair Schwarz Approximation

The omitted match-count terms suggest a richer approximation based on smaller, overlapping problems. Additive Schwarz preconditioning adds corrections for the worker-firm, worker-year, and firm-year pairs (Xu 1992; Toselli and Widlund 2005). Each local problem retains its match-count block, while the outer LSMR iteration handles the remaining three-way coupling and any error in the local correction.

The worker-firm local problem is

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \quad (21)$$

Figure 5 compares the count-only worker-firm block used by diagonal preconditioning with the factor-pair block.

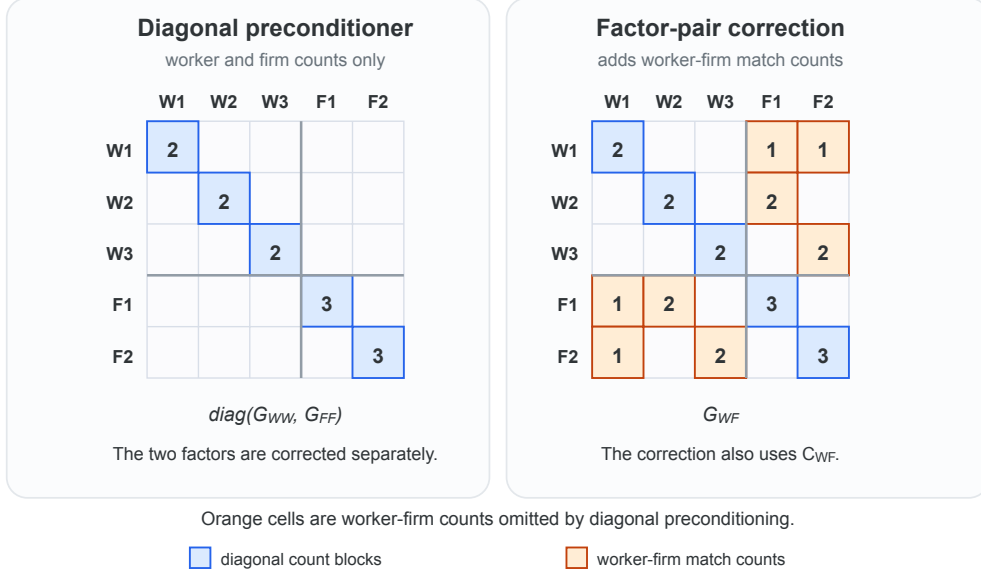


Figure 5: Count-only and factor-pair corrections for the example from Section 4. The diagonal preconditioner (left) treats the blue worker and firm count blocks separately, whereas the factor-pair correction (right) retains those blocks and adds the orange worker-firm observation counts C_{WF} .

The inverse of the worker-firm block retains the match links through the Schur complement. To place this correction in the three-factor problem, let R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}^T$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ weights levels that appear in more than one pair problem with partition-of-unity weights, so their corrections are not counted twice. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'} \tilde{D}_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix}^{-1} \tilde{D}_{WF} R_{WF}. \quad (22)$$

The worker-year and firm-year terms have the same form. Because each level belongs to two pair problems, its weight in each is $\frac{1}{\sqrt{2}}$, which makes the squared weights sum to one. Adding the three terms gives the exact factor-pair Schwarz preconditioner,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \quad (23)$$

The pair terms capture all three cross-tabulations separately, while LSMR resolves the joint three-factor coupling and the error from splitting shared levels across pairs.

6.5. Approximating Pair Systems via Graph Laplacians

After one normalization, the five-level worker-firm block in Section 4 becomes a 4×4 system, but register data can contain hundreds of thousands of workers and firms, making direct inversion impractical. We approximate large blocks using their graph-Laplacian structure.

For a worker-firm pair, the local pair step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix} x = u, \quad (24)$$

where u contains the selected and weighted worker and firm entries of the LSMR vector. Since the match counts C_{WF} are non-negative, we obtain a Laplacian by using $T_{WF} = \text{diag}(I_W, -I_F)$ to flip the firm signs:

$$L_{WF} = T_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (25)$$

The resulting matrix is a weighted bipartite graph Laplacian with non-positive off-diagonals and zero row sums. Since $T_{WF}^2 = I$, the same sign change maps its inverse back to the pair Gramian,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} = T_{WF} L_{WF}^{-1} T_{WF}, \quad (26)$$

where all inverses use the normalization from Section 6.1. Within each connected component, the implementation returns the zero-mean solution, thereby fixing the free constant without changing the residualized variables.

Because LSMR refines the remaining error, the local correction need not be exact; we use sparse approximate Cholesky factorizations (Spielman and Teng 2014; Gao et al. 2025). Exact elimination can create fill-in as removing a worker links the firms that employed that worker and later eliminations add more entries, so a dense k -level factorization can require order k^3 operations and k^2 memory. Randomized approximation limits this growth and has cost close to the number of observed links up to logarithmic factors. We denote the resulting approximate pair-Gramian inverse, after changing the signs back, by A_{WF} .

The worker-year and firm-year approximations are A_{WY} and A_{FY} . Substituting all three approximate inverses into the Schwarz sum gives the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \quad (27)$$

The exact P^{-1} replaces the full three-factor inverse with exact pair inverses, while M^{-1} also approximates each pair inverse through A_{qr} . Both approximations change the update at each LSMR iteration but leave the original fixed-effect least-squares problem unchanged. LSMR therefore returns the same fitted residuals as an unpreconditioned LSMR run, up to the requested tolerance.

6.6. Implementation Strategy

Figure 6 retains the outer LSMR loop from Figure 4 and replaces the count-based correction with three factor-pair corrections. After the sign change, sparse approximate Cholesky provides a local inverse for each pair graph; partition-of-unity weights then combine the corrections without counting shared levels twice.

LSMR applies this sum while solving the original least-squares problem (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024). When observations, weights, and fixed-effect identifiers are unchanged, the pair factorizations can be reused for the outcome and every covariate. The `within` implementation follows this setup-and-apply sequence (within Developers 2026).

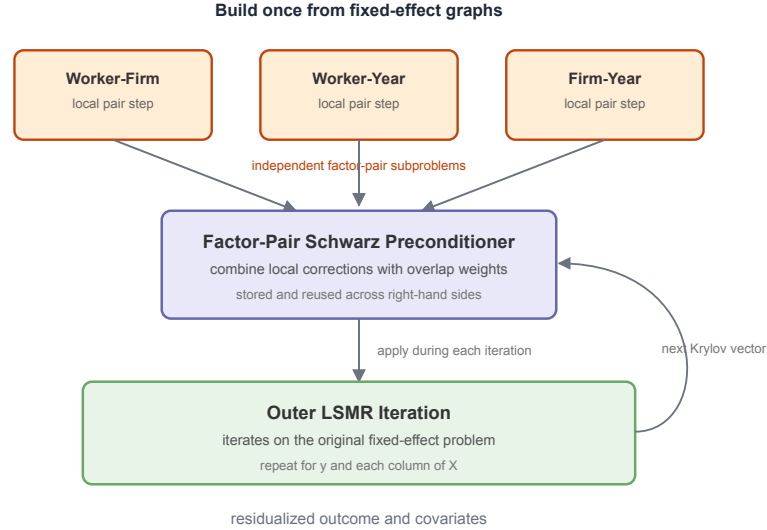


Figure 6: Factor-pair preconditioning within LSMR. Each local problem combines two fixed-effect dimensions and retains their observed links; a sign change then turns the pair matrix into a graph Laplacian, for which sparse approximate Cholesky supplies a local correction. Partition-of-unity weights combine the corrections before they enter the same outer LSMR iteration shown in Figure 4.

7. Benchmarks

The benchmarks ask when fewer iterations repay the cost of building the factor-pair preconditioner. We vary mobility and sorting in controlled AKM panels, compare two public synthetic designs, and use eight real datasets from Correia (2017).

Elapsed times cover the package-level regression call, from model setup through coefficient estimation. Cross-package comparisons use the same input data but retain each package’s default treatment of singleton levels and separated observations, whereas experiments that isolate the preconditioner use one prepared sample. The appendix reports the additional memory used to store the pair structures.

Each table reports the factor-pair Gap after iterative singleton removal. For a disconnected graph, the value comes from the component with the most retained observations, whose observation share appears in parentheses. As in Section 5, the Gap remains a pairwise diagnostic in models with three or more fixed effects.

The OLS package-runtime tables compare six configurations: PyFixest MAP, PyFixest LSMR with no, diagonal, or factor-pair preconditioning, R fixest, and FixedEffectModels.jl:

Configuration	Package	Algorithm
rust-map	PyFixest	Vanilla Rust MAP, without acceleration.
within-off	PyFixest	LSMR without preconditioning.
within-diagonal	PyFixest	LSMR with diagonal preconditioning.
within	PyFixest	LSMR with factor-pair Schwarz preconditioning.
fixest	R fixest	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
FEM.jl	FixedEffectModels.jl	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).

Tables report medians over the stated number of repetitions. All runs use an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1. We omit `reghdfe` (Correia 2014; 2017) because we lack a Stata license; its accelerated MAP algorithm is represented here by `fixest`.

7.1. Runtime Benchmarks

Package defaults are useful to users but do not impose equal accuracy because the implementations apply tolerances to different quantities. We report both package-default times and a comparison based on achieved coefficient and residual error.

The main OLS panels contain one million observations, one covariate, ten periods, and worker, firm, and year effects. One set of designs lowers the probability of moving, while the other holds that probability at one but increasingly assigns moves within groups of similar firms.

7.1.1. Worker Mobility

As mobility declines, the worker-firm Gap falls from 0.232 to 2.41×10^{-5} ; all configurations finish within 3.60 seconds in the first two designs. PyFixest MAP then slows sharply or reaches its iteration cap, and unpreconditioned LSMR also reaches the cap. Factor-pair LSMR falls from 0.554 seconds in the first design to 0.373 seconds in the last because the local worker-firm graphs become cheaper to factorize.

7.1.2. Sorting Among Movers

Although every worker moves between periods, increasing ρ concentrates moves within groups, and the Gap falls from 0.222 to 2.13×10^{-4} . PyFixest MAP rises from 0.411 seconds to 53.9 seconds before reaching its cap in the final design; unpreconditioned LSMR reaches its cap in the last four. Over the same designs, diagonal LSMR rises from 0.362 to 1.58 seconds, while factor-pair LSMR stays below one second and is faster than diagonal LSMR at the three smallest Gap values.

7.1.3. When Preconditioner Setup Dominates

The public `fixest` benchmark suite complements the gradual connectivity changes in the AKM designs by contrasting a dense, well-connected design with a sparse, nearly nested design (Bergé et al. 2026). Both contain 10 million observations, one covariate, and worker, firm, and year fixed effects.

Simple and difficult `fixest` designs (10 million observations, three fixed effects).

Design	Gap λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
simple (well-connected)	0.622 (1.00)	2.35s	2.72s	2.16s	11.2s	2.52s	2.08s
difficult (near-nested)	8.35×10^{-8} (1.00)	326.2s	10.2s	capped (0/3)	4.63s	62.2s	27.0s

Note: Times are in seconds and cover each OLS regression from model setup through coefficient estimation. Each package uses its default settings. A preliminary run sets the number of measured runs: 20 if it takes less than 1 second, 7 if it takes 1 to 10 seconds, and 3 if it takes longer. When only k of n planned runs return an estimate, $t(\frac{k}{n})$ reports their median time t . A dash means that no timing is available. capped (0/ n) means that no run finishes before the iteration limit; failed (0/ n) marks a failure other than reaching that limit. Both designs have 10 million observations, one covariate, and worker, firm, and year fixed effects. Gap λ_2 is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

The dense design exposes the setup cost: five methods finish in 2.08 to 2.72 seconds, while factor-pair LSMR takes 11.2 seconds. Near nesting reverses the ranking. Factor-pair LSMR takes 4.63 seconds,

compared with 10.2 seconds without preconditioning, 27.0 seconds for FEM.jl, 62.2 seconds for `fixest`, and 326.2 seconds for PyFixest MAP; diagonal LSMR reaches its default cap.

7.1.4. Iterations After Setup

To separate iteration counts from setup time, we use 100,000-observation versions of the same designs.

Iterations on the simple and difficult designs (100,000 observations).

Design	map (passes)	off	diagonal	additive
simple	14	37	16	14
difficult	8159	249	180	22

Note: The MAP column counts complete passes over all fixed-effect dimensions. The other columns count LSMR iterations: `off` uses no preconditioner, `diagonal` scales by fixed-effect group counts, and `additive` uses the factor-pair preconditioner. A MAP pass and an LSMR iteration perform different work, so their counts are not directly comparable. Each count is the median of three runs; each run removes the fixed effects from one outcome and one covariate.

Factor-pair LSMR needs 14 iterations on the simple design and 22 on the difficult one. The corresponding counts rise from 16 to 180 for diagonal LSMR and from 37 to 249 without preconditioning. The factor-pair iteration count is stable, but its setup still makes it unattractive for the simple one-shot regression.

7.1.5. Public Real-Data Benchmarks

The controlled designs isolate mobility, sorting, and near nesting, but empirical fixed-effect graphs also contain large differences in group size, thin links between dense groups, and many disconnected components. The public benchmark collection assembled by Correia (2017) contains eight such datasets and lets us check whether the controlled comparisons carry over to these less regular graphs.

Correia real-data benchmarks.

Dataset	Gap λ_2 (share)	PyFixest MAP	PyFixest LSMR factor-pair	fixest	FEM.jl
credit	0.227 (1.00)	0.193s	0.242s	0.154s	0.097s
soccer	0.666 (1.00)	0.026s	0.065s	0.014s	0.010s
enron	3.53×10^{-3} (0.99)	2.95s	0.402s	0.535s	0.317s
github	3.15×10^{-4} (0.32)	capped (0/3)	0.310s	1.43s	1.44s
patents	2.59×10^{-4} (0.95)	capped (0/3)	0.388s	1.05s	0.576s
workers	1.37×10^{-4} (0.63)	capped (0/3)	0.279s	2.00s	1.45s
schools	1.11×10^{-3} (1.00)	5.98s	0.266s	0.828s	0.487s
directors	2.56×10^{-4} (0.30)	capped (0/3)	0.327s	0.283s	0.462s

Note: Times are in seconds for OLS regressions using each package’s default settings, with three planned runs per cell. Each package applies its own rule for removing fixed-effect levels observed only once. If only k runs return an estimate, $t(\frac{k}{3})$ reports their median time t . `capped (0/3)` means that no run finishes before the 10,000-iteration limit. Gap λ_2 is computed after iterative singleton removal on the `id1-id2` component containing the most retained observations; its observation share appears in parentheses.

On `credit` and `soccer`, PyFixest MAP takes 0.193 and 0.026 seconds, compared with 0.242 and 0.065 seconds for factor-pair LSMR, so the preconditioner’s setup cost does not pay on these easy datasets. Factor-pair LSMR is faster on the more difficult `enron` and `schools` graphs, taking 0.402 and 0.266 seconds compared with 2.95 and 5.98 seconds for PyFixest MAP. PyFixest MAP reaches its iteration cap on `github`, `patents`, `workers`, and `directors`, while factor-pair LSMR finishes in 0.279 to 0.388 seconds. Accelerated MAP can still be competitive, however; `fixest` takes 0.283 seconds on `directors`, slightly less than factor-pair LSMR’s 0.327 seconds.

The Gap is a pairwise description rather than a rule for choosing a solver because runtime also depends on component size, factor dimensions, graph sparsity, acceleration, and the preconditioner’s setup cost.

7.2. Runtime and Achieved Precision

Solver comparisons also depend on how packages define convergence. MAP implementations compare changes in residuals or fixed-effect coefficients, while LSMR uses its estimated least-squares residual and first-order-condition error (Fong and Saunders 2011), so their numerical tolerances are not a common measure of accuracy. For example, within stops when either its relative residual or its scaled normal-equation error meets the requested threshold; we compute the latter as

$$\frac{\|A^T r\|_2}{\|A\|_F \|r\|_2}, \quad A = W^{\frac{1}{2}} D, \quad r = W^{\frac{1}{2}} (\mu - D\alpha). \quad (28)$$

Figure 7 places the methods on a common accuracy scale for mobility designs 1, 3, and 5. Every method receives the same singleton-pruned sample and a common 10,000-iteration cap. We time three fits at each package-specific tolerance and compare their coefficients and residuals with factor-pair LSMR at tolerance 10^{-14} .

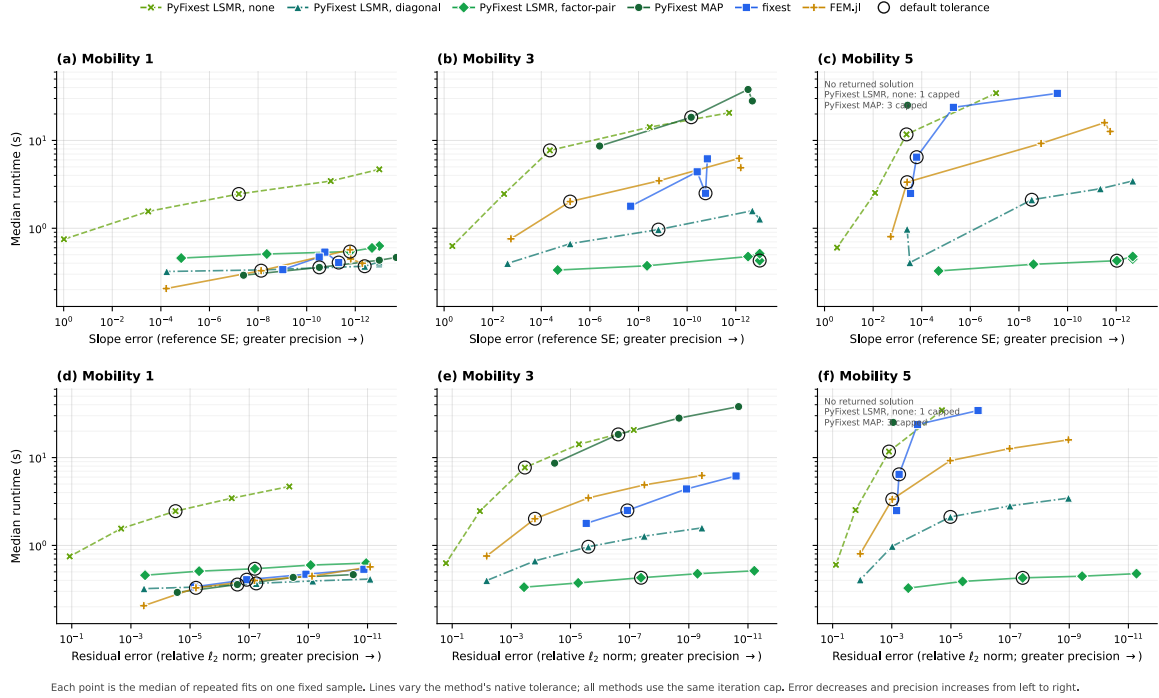


Figure 7: Elapsed time and achieved accuracy on three AKM mobility designs. Each marker reports the median elapsed time among the three planned fits that return an estimate on the same sample. Before timing, we remove fixed-effect levels observed only once and use the remaining observations for every method. Lines connect results for one method across requested tolerances. The top row measures coefficient error as $\frac{\|\hat{\beta} - \beta^*\|_2}{\text{SE}(\hat{\beta}^*)}$, and the bottom row measures residual error as $\frac{\|r - r^*\|_2}{\|r^*\|_2}$; the reference coefficient $\hat{\beta}^*$ and residual r^* come from factor-pair LSMR at tolerance 10^{-14} . Error decreases from left to right, and a circle on each line marks the result obtained with the package’s default tolerance. Every fit is limited to 10,000 iterations; settings for which none of the three fits returns an estimate are omitted and identified in the annotations.

7.3. Amortizing the Preconditioner

Matching achieved accuracy does not remove the factor-pair setup cost, but later regressions can reuse the pair blocks and factorizations when the observations, weights, and fixed-effect identifiers are unchanged.

7.3.1. Setup Cost Across Connectivity

We separate construction from LSMR iterations on the one-million-observation mobility designs, for specifications with worker-firm and worker-firm-year effects.

Factor-pair setup and LSMR times as worker mobility varies.

Move probability δ	Gap λ_2 (share)	2 FE setup	2 FE LSMR	3 FE setup	3 FE LSMR
1	0.232 (1.00)	0.109s	0.291s	0.113s	0.209s
0.5	0.0392 (1.00)	0.115s	0.255s	0.118s	0.234s
0.05	1.84×10^{-3} (0.87)	0.032s	0.135s	0.034s	0.233s
0.01	5.35×10^{-4} (0.61)	0.027s	0.124s	0.031s	0.216s
0.005	5.50×10^{-5} (0.52)	0.025s	0.093s	0.031s	0.193s
0.001	2.41×10^{-5} (0.29)	0.028s	0.045s	0.032s	0.149s

Note: Times are in seconds. For each design and specification, we plan five runs with 1 million observations and an LSMR tolerance of 10^{-12} . The table gives the median setup time and time spent in LSMR among the runs that finish; if only k finish, $t(\frac{k}{5})$ reports their median time t . The two-fixed-effect specification absorbs worker and firm effects; the three-fixed-effect specification also absorbs year effects. Gap λ_2 is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

From the highest- to the lowest-mobility design, setup falls from 0.109 to 0.028 seconds with two fixed effects and from 0.113 to 0.032 seconds with three; time spent in LSMR also falls. The low-mobility graph has fewer cross-firm links to store and factorize, which explains the declining factor-pair runtime in Figure 1.

7.3.2. Ten Regressions on the Same Fixed Effects

Empirical projects often estimate several specifications on one sample and set of fixed effects. We time ten sequential regressions on both one-million-observation fixed designs, comparing diagonal preconditioning with factor-pair preconditioning that is rebuilt for each regression or retained for all ten.

Ten regressions with rebuilt and cached preconditioners.

Design	Policy	Setup	LSMR	Total	Speedup
simple	Diagonal	0.048s	0.553s	0.601s	1.0x
	Additive, rebuilt	2.80s	1.51s	4.29s	0.1x
	Additive, cached	0.281s	1.51s	1.79s	0.3x
difficult	Diagonal	0.049s	43.6s	43.6s	1.0x
	Additive, rebuilt	0.379s	1.20s	1.58s	27.5x
	Additive, cached	0.040s	1.26s	1.30s	33.6x

Note: For each policy, we add the setup and LSMR times across ten sequential regressions and take the median across three planned repetitions. If only k repetitions finish, $t(\frac{k}{3})$ reports their median time t , and the speedup is omitted. Times are in seconds. Each design has 1 million observations and worker, firm, and year fixed effects. Each regression removes the fixed effects from the common outcome and one of ten covariates at an LSMR tolerance of 10^{-12} . For complete cells, speedup divides the diagonal total time by the reported total time. Additive, rebuilt constructs a new factor-pair preconditioner for each regression; Additive, cached reuses one preconditioner for all ten.

Reuse lowers factor-pair time on the simple design from 4.29 to 1.79 seconds, but diagonal preconditioning still takes only 0.601 seconds. On the difficult design, the ranking is decisive: ten regressions take 43.6 seconds with diagonal preconditioning, 1.58 seconds when the factor-pair preconditioner is rebuilt, and 1.30 seconds when it is retained.

7.4. Poisson and Other GLMs

Reuse also matters within a single nonlinear fit because Poisson fixed-effect models repeat the absorption step at every iteratively reweighted least-squares (IRLS) iteration (Stammann 2018). Their fixed-effect identifiers remain fixed while the weights and working response change, so `ppmlhdfc` (Correia et al. 2020)

and related estimators repeatedly residualize the data with the same identifiers and accumulate this cost over the fit.

PyFixest currently retains the preconditioner from the first weighted demeaning call as the IRLS weights change. The benchmark reports that policy; it does not isolate reuse as the cause of the runtime difference. The comparison uses the one-million-observation simple and difficult fixest designs (Bergé et al. 2026). It includes R `fixest::fepois`, `GLFixedEffectModels.jl`, and PyFixest `fepois` with MAP or factor-pair LSMR. Each implementation receives the same 100-iteration IRLS limit and applies its default separation rule.

Poisson benchmarks (1 million observations, one covariate).

Design	FE	PyFixest MAP	fixest	GLFEM.jl	PyFixest LSMR factor-pair
simple (well-connected)	3	7.95s	4.63s	5.71s	9.74s
difficult (near-nested)	3	capped (0/3)	439.3s	129.8s	5.52s

Note: Times are in seconds for Poisson fixed-effect regressions with 1 million observations, one covariate, and worker, firm, and year fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` use PyFixest `fepois`; and `GLFEM.jl` is `GLFixedEffectModels.jl`. The `within` configuration reuses the first factor-pair preconditioner as the weights change across iteratively reweighted least squares steps. Each package applies its default rule for removing separated observations. If only k of the three planned fits return an estimate, $t(\frac{k}{3})$ gives their median time t . `capped (0/3)` means that no fit finishes before either the package’s inner demeaning limit or the common limit of 100 iteratively reweighted least squares steps; `failed (0/3)` marks another failure.

All four implementations finish the simple design in less than ten seconds. On the difficult design, PyFixest MAP reaches its iteration cap, `GLFEM.jl` takes 129.8 seconds, `fixest` takes 439.3 seconds, and factor-pair LSMR takes 5.52 seconds. Repeated absorption magnifies the cost of slow propagation across the worker-firm graph.

8. Software

The benchmarks use the open-source `within` solver (within Developers 2026), whose computational core is written in Rust and exposed through Python and R APIs.

Interface	Package	Registry	Install
Rust	<code>within</code>	<code>crates.io</code>	<code>cargo add within</code>
Python	<code>within-py</code>	PyPI	<code>pip install within-py</code>
R	<code>withinr</code>	<code>py-econometrics R-universe</code>	<code>install.packages("withinr", repos = "https://py-econometrics.r-universe.dev")</code>

PyFixest users can select the method without constructing the Gramian or its pair blocks (PyFixest Developers 2026):

```
import pyfixest as pf

fit = pf.feols(
    "y ~ x1 + x2 | worker + firm",
    data=df,
    demeaner=pf.LsmrDemeaner(),
)
```

The lower-level Python interface exposes the FWL operation, with `worker_code` and `firm_code` supplied as zero-based integer codes:

```
import numpy as np
from within import solve_batch
```

```
categories = np.asfortranarray(
    df[["worker_code", "firm_code"]].to_numpy(dtype=np.uint32)
)
variables = np.asfortranarray(df[["y", "x1", "x2"]].to_numpy())
res = solve_batch(categories, variables)
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

`solve_batch` residualizes all three columns with one factor-pair preconditioner, after which the last line runs the low-dimensional FWL regression from Section 2.

9. Conclusion

MAP's low cost per pass makes it effective for many well-connected fixed-effect models, but successive mean updates move information slowly when mobility is sparse, sorting is strong, or one effect is nearly nested in another.

Factor-pair preconditioning uses observed co-occurrences to correct worker-firm and other pair blocks jointly, changing the numerical update while preserving the original least-squares target. Its setup cost can outweigh the reduction in iterations on easy graphs, but on difficult graphs it can avoid hundreds of MAP passes or LSMR iterations; sparse pair graphs can also be cheaper to factorize than dense ones.

The setup cost is most likely to be worthwhile when the factor-pair Gap is small or the same fixed effects are absorbed repeatedly. Specification searches can reuse one preconditioner across outcomes and covariates, while IRLS estimators call the demeaning routine at every outer iteration. In both cases, reuse spreads the setup cost across several residualizations. The Gap remains a pairwise connectivity diagnostic rather than a solver-selection rule, but a small value signals the weak connectivity that can slow MAP.

Appendix: Memory Use

MAP stores the current residuals and group sums, whereas factor-pair LSMR also retains match structures and local factorizations. We compare peak resident memory for the two Rust implementations inside PyFixest, using isolated processes so the surrounding regression code is held fixed.

Peak physical memory use (three fixed effects, one covariate).

Design	Gap λ_2 (share)	PyFixest MAP	PyFixest LSMR factor-pair
<i>100K observations</i>			
simple (well-connected)	0.622 (1.00)	354 MiB	411 MiB
difficult (near-nested)	6.51×10^{-4} (1.00)	357 MiB	383 MiB
<i>1M observations</i>			
simple (well-connected)	0.622 (1.00)	758 MiB	1,000 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	775 MiB	953 MiB

Note: For each regression, we record the largest amount of physical memory used by an isolated Python process, in MiB (2^{20} bytes). The table compares PyFixest OLS regressions using MAP or factor-pair LSMR. Each regression has one covariate and worker, firm, and year fixed effects. The results cover the simple and difficult designs at 100,000 and 1 million observations. Gap λ_2 is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

Factor-pair LSMR uses slightly more memory: 26-57 MiB at 100K observations and 178-242 MiB at 1M; the additional storage holds the pairwise co-occurrences, partition weights, and approximate Cholesky factors.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. <https://doi.org/10.1088/0266-5611/30/7/075009>.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R." <https://doi.org/10.48550/arXiv.2601.21749>.
- Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects." <https://github.com/sergiocorreia/reghdfe>.
- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. <https://doi.org/10.1177/1536867X20909691>.
- FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables." <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*. <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields*. Technical report.
- Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.
- Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. <https://doi.org/10.1002/nme.1620010306>.
- Jochmans, Koen, and Martin Weidner. 2019. "Fixed-Effect Regressions on Network Data." *Econometrica* 87 (5): 1543–60. <https://doi.org/10.3982/ECTA14605>.

- Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python." <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects." <https://doi.org/10.48550/arXiv.1707.01815>.
- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. <https://doi.org/10.1007/b137868>.
- within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data." <https://github.com/py-econometrics/within>.
- Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.
- Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. "Flexible Modified LSMR for Least Squares Problems."